Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes

# TP : IDM – De M3 à M0 avec EMF

Méta-modèle Ecore, modèle XMI, contraintes OCL, exécution et génération

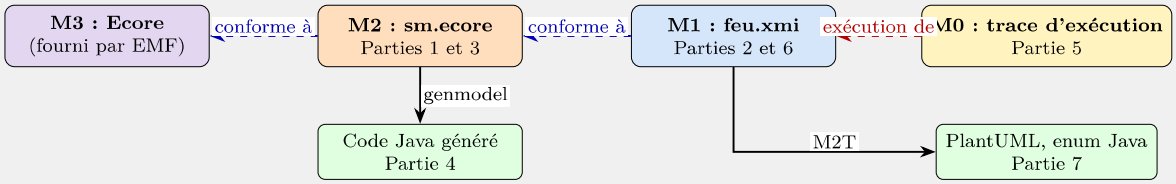
***(Énoncé)***

**Pr.Aissam BERRAHOU**

*Année universitaire 2026–2027 – durée : 2 séances de 3 h*

## Objectifs, outils et livrables

**Objectif général.** Parcourir **concrètement** la pile de l'OMG vue en cours, avec le fil rouge du **feu de signalisation** : on écrit un méta-modèle (M2) avec Ecore (M3), on crée des modèles (M1) qui lui sont conformes, on les vérifie avec OCL, puis on les **exécute** pour observer des comportements réels (M0) et on les **transforme** en texte.




**Outils.** Eclipse **Modeling Tools** (version récente) avec : EMF et Ecore Tools (inclus), **OCLinEcore** (*Eclipse OCL*, via *Help* > *Install Modeling Components*), JDK 17 ou plus. Optionnel : Acceleo 3 (Partie 7), un visualiseur PlantUML.
**Ressources fournies** (dossier `ressources/`) : `sm.ecore`, `sm.oclinecore`, `model/feu.xmi`, `model/feupieton.xmi`, `src/fr/ensias/idm/simulation/*.java`. Les fichiers XMI et Java ne sont à utiliser qu'à partir des parties indiquées (ils servent aussi de corrigé de référence).
**Livrables.** Une archive `NOM_Prenom_TP_IDM.zip` contenant les projets Eclipse et un compte rendu PDF (3 à 5 pages) avec les captures demandées dans les encadrés « À rendre ».

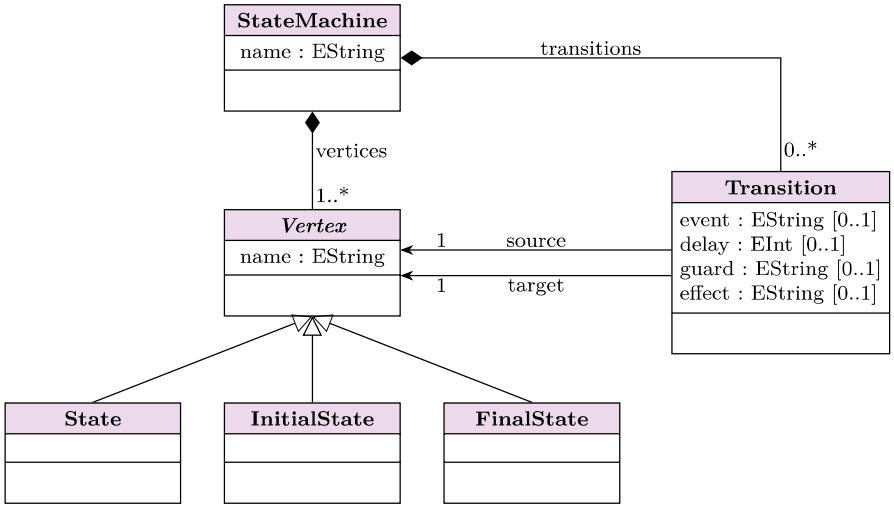
| Partie | Contenu | Niveau |
|---|---|---|
| 0 | Préparer l'environnement | – |
| 1 | Construire le méta-modèle des machines à états | M2 (avec M3) |
| 2 | Créer les modèles du feu et du feu piéton | M1 |
| 3 | Vérifier la conformité ; ajouter des contraintes OCL | M1 / M2 |
| 4 | Générer le code Java du méta-modèle | M2 → code |
| 5 | Exécuter un modèle : le simulateur | M1 → M0 |
| 6 | Construire un modèle par programme | M1 |
| 7 | Transformation modèle vers texte (M2T) | M1 → texte |
| 8 | Pour aller plus loin : le DSL du tramway | M2, M1, M0 |

## Partie 0 : Préparer l'environnement [Simple]

1. Lancer Eclipse Modeling Tools avec un nouvel espace de travail `ws-idm-2027`.
2. Installer OCLinEcore : *Help* > *Install Modeling Components* > *OCL*, puis redémarrer.
3. Créer un projet **Ecore Modeling Project** nommé `fr.ensias.idm.sm`, package `sm`, avec le point de vue *Design*. Observer l'arborescence : dossier `model/` avec `sm.ecore` et `sm.aird` (représentation graphique).
4. Question : dans l'onglet *Properties*, quelles sont les propriétés de l'EPackage `sm` ? À quoi servent `nsURI` et `nsPrefix` ?

## Partie 1 : Construire le méta-modèle des machines à états (M2) [Simple]

On implante, avec Ecore, une version du méta-modèle de la machine à états vu en cours, adaptée à l'exécution :




1. Avec l'éditeur graphique (*Design*) ou l'éditeur arborescent, créer les **EClass** `StateMachine`, `Vertex` (`abstract = true`), `State`, `InitialState`, `FinalState` et `Transition`, puis les **héritages** (`eSuperTypes`).
2. Ajouter les **EAttribute** avec leurs types (`EString`, `EInt`) et leurs multiplicités.

3. Ajouter les **EReference**. Pour chacune, choisir correctement `containment`, `lowerBound` et `upperBound` ($-1$ = « * »).
4. Valider le méta-modèle (clic droit sur l'EPackage > *Validate*). Puis ouvrir `sm.ecore` avec un éditeur de texte : à quoi ressemble un méta-modèle sur le disque ?
5. [M2 / M3] Compléter le tableau.

| Élément de sm.ecore (M2) | Instance de (M3) | Propriétés importantes |
|---|---|---|
| `Transition` | | |
| `Vertex` | | |
| `delay` | | |
| `vertices` | | |
| `source` | | |
| `EString` | | |
| `sm` | | |

**À rendre**

Capture du diagramme Ecore et le tableau complété. Expliquer en deux phrases pourquoi `vertices` est une composition mais pas `source`.

## Partie 2 : Créer les modèles du feu et du feu piéton (M1) [Simple]

1. Dans `sm.ecore`, clic droit sur l'EClass `StateMachine` > *Create Dynamic Instance...*, fichier `model/feu.xmi`. On obtient un modèle M1 **sans avoir généré de code** : EMF utilise le méta-modèle de manière *réflexive*.
2. Avec l'éditeur arborescent, construire le modèle du feu du cours : un état initial `debut`, les états `Rouge`, `Vert`, `Orange`, un état final `fin`, et les transitions : `debut` → `Rouge` ; `Rouge` → `Vert` (`delay = 25`) ; `Vert` → `Orange` (20) ; `Orange` → `Rouge` (5) ; `Rouge` → `fin` (`event = eteindre`).
3. Ouvrir `feu.xmi` avec un éditeur de texte. Comment sont représentés : le type d'un sommet ? une référence vers un état ?
4. Créer de même `model/feupieton.xmi` pour le feu piéton du TD (exercice 3) : la transition `PietonRouge` → `PietonVert` a `event = bouton`, `guard = tempsDansEtat >= 30`, `effect = bip()`.

**À rendre**

Capture de l'éditeur arborescent du modèle `feu.xmi` et réponse à la question 3.

## Partie 3 : Vérifier la conformité ; ajouter des contraintes OCL [Moyen]

**3.1 – Conformité structurelle.**

1. Dans `feu.xmi`, clic droit sur la racine > *Validate*. Le modèle doit être valide.
2. Introduire volontairement les erreurs suivantes, valider après chacune, noter le message, puis corriger : (a) supprimer la cible d'une transition ; (b) ajouter un second `InitialState` ; (c) ajouter une transition qui part de `fin`.
3. Lesquelles de ces erreurs sont détectées ? Pourquoi les autres ne le sont-elles pas ?

**3.2 – Contraintes OCL.** Le méta-modèle seul ne peut pas exprimer toutes les règles du cours. On les ajoute avec **OCLinEcore** : clic droit sur `sm.ecore` > *Open With* > *OCLinEcore Editor*.

4. Écrire les invariants suivants : **(I1)** une machine possède exactement un état initial ; **(I2)** les noms des sommets d'une machine sont uniques ; **(I3)** aucune transition ne sort d'un état final ; **(I4)** aucune transition n'entre dans un état initial ; **(I5)** toute transition, sauf celle qui part de l'état initial, a un déclencheur (un événement ou un délai strictement positif).
5. Enregistrer, puis refaire les tests (b) et (c) de la question 2. Que constatez-vous ?

**Astuce**

Dans l'éditeur OCLinEcore, un invariant s'écrit dans le corps de la classe : `invariant Nom: expression-booléenne;`. Les opérations utiles sont `select`, `size`, `isUnique`, `oclIsKindOf`.

**À rendre**

Le texte OCLinEcore complet et une capture de la vue *Problems* pour le test (b).

## Partie 4 : Générer le code Java du méta-modèle [Simple]

1. Clic droit sur `sm.ecore` > *New* > *EMF Generator Model* (`sm.genmodel`). Dans les propriétés du package `Sm`, fixer `Base Package = fr.ensias.idm`.
2. Clic droit sur la racine du genmodel > *Generate Model Code*, puis *Generate Edit Code* et *Generate Editor Code*.
3. Explorer le code produit dans `src-gen/` (ou `src/`) et répondre :
   (a) Quels fichiers Java correspondent à l'EClass `Transition` ? Que contient l'interface ?
   (b) Quel est le rôle de `SmFactory` et de `SmPackage` ?
   (c) Que devient un attribut `EInt [0..1]` ? une référence `[1..*]` ?
   (d) Que signifie l'annotation `@generated` ? Comment conserver une modification manuelle lors d'une nouvelle génération ?
4. Lancer une *Eclipse Application* (Run As) et créer un modèle avec l'éditeur généré *Sm Model*.

**À rendre**

Les réponses (a) à (d) et une capture de l'éditeur généré.

## Partie 5 : Exécuter un modèle – le simulateur (M1 → M0) [Moyen]

Un modèle M1 décrit **tous** les feux possibles. Pour obtenir le comportement d'**un** feu réel (M0), on *exécute* le modèle : c'est le rôle d'un simulateur.

1. Dans un nouveau projet Java (dépendances : le projet `fr.ensias.idm.sm` et les plug-ins `org.eclipse.emf.ecore`, `org.eclipse.emf.ecore.xmi`), créer la classe `fr.ensias.idm.simulation.Simulateur`. Copier le dossier `model/` à la racine du projet.
2. Écrire la méthode `charger(String chemin)` qui charge un fichier XMI et renvoie la `StateMachine` racine.
3. Écrire le constructeur : il cherche l'`InitialState` et franchit sa transition sortante.
4. Écrire `avancerJusqua(int t)` : tant que l'état courant n'est pas final, chercher la transition temporelle sortante la plus proche (`delay > 0`), la franchir si elle a lieu avant `t`. Chaque franchissement affiche une ligne de trace : `t = 25 s | Rouge -after(25 s)-> Vert`.
5. Écrire `evenement(String nom, int t)` : avancer jusqu'à `t`, puis franchir la transition sortante déclenchée par `nom` si sa garde est vraie. On n'interprète que les gardes de la forme `tempsDansEtat >= N` (temps passé dans l'état courant).
6. Exécuter les deux scénarios suivants et recopier les traces :
   - `feu.xmi` : événement `eteindre` à t = 110 s, fin à 140 s ;
   - `feupieton.xmi` : `bouton` à t = 10 s puis à t = 35 s, fin à 80 s.
7. Que se passe-t-il si l'on envoie `eteindre` à t = 130 s ? Expliquer à l'aide du modèle M1.

**À rendre**

Le code du simulateur, les deux traces et la réponse à la question 7. Indiquer clairement, dans le compte rendu, ce qui relève de M1 (le fichier XMI) et ce qui relève de M0 (la trace).

## Partie 6 : Construire un modèle par programme (M1) [Moyen]

1. Écrire la classe `Constructeur` qui crée, **uniquement avec** `SmFactory`, le modèle d'un feu orange clignotant : `debut` → `Eteint` ; `Eteint` → `Allume` (`delay = 1`) ; `Allume` → `Eteint` (1) ; `Eteint` → `fin` (`event = arret`).
2. Valider le modèle avec `Diagnostician.INSTANCE.validate(...)` puis l'enregistrer dans `model/clignotant.xmi`.
3. Ouvrir le fichier produit dans l'éditeur arborescent, puis le simuler (`arret` à t = 4 s, fin à 6 s).
4. Question : quelle différence y a-t-il entre ce programme et le simulateur du point de vue des niveaux de l'OMG ?

## Partie 7 : Transformation modèle vers texte (M2T) [Moyen]

Une transformation M2T lit un modèle et produit du texte. Ses règles sont écrites **une seule fois** au niveau du méta-modèle et s'appliquent à **tous** les modèles conformes.

1. Écrire la classe `GenerateurPlantUML` qui produit, pour une machine à états, (a) un fichier `gen/<nom>.puml` décrivant le diagramme d'états en syntaxe PlantUML (`[*] -> Rouge`, `Rouge -> Vert : after(25 s)`…) et (b) une énumération Java `Etats<nom>` contenant les états (hors initial et final).
2. L'appliquer à `feu.xmi`, `feupieton.xmi` et `clignotant.xmi`. Visualiser les `.puml` (extension PlantUML ou site plantuml.com).
3. **(Optionnel)** Écrire la même transformation avec un *template* Acceleo.
4. Question : comparer cette génération avec celle de la partie 4. Qu'est-ce qui est généré, et à partir de quel niveau ?

## Partie 8 : Pour aller plus loin – le DSL du tramway [Avancé]

On reprend l'exercice 9 du TD (réseau de tramway Rabat–Salé).

1. Créer le projet `fr.ensias.idm.tram` et implanter le méta-modèle (`Reseau`, `Station`, `Ligne`, `Troncon`, `Rame`) en Ecore, avec `ordered = true` pour les tronçons.
2. Ajouter en OCLinEcore les contraintes : noms de stations uniques ; enchaînement des tronçons ; terminus aux extrémités ; durée strictement positive.
3. Créer une instance dynamique `rabat.xmi` avec la ligne L1 réduite (Agdal → Bab El Had → Salé Tabriquet) et vérifier qu'elle est conforme ; introduire une rupture d'enchaînement et observer le message.
4. Écrire un générateur (Java ou Acceleo) qui produit, pour chaque ligne, une fiche horaire HTML : station, temps cumulé depuis le premier terminus.

*— Bon travail —*